Assignment1

1. Trace the historical evolution of AI from ELIZA to modern Generative AI models.
2. Contrast Classical Machine Learning with Deep Learning in terms of feature engineering, representation learning, and manual abstraction.
3. Traditional machine learning models require manual feature, whereas Deep Neural Networks extract hierarchical features automatically from raw inputs.
4. What is the difference between a list and a tuple in Python?
5. What is NumPy used for in Python?Name any two functions used in Pandas for data manipulation.
6. What is the purpose of Matplotlib in data science?
7. What is FastAPI, and why is it used?
8. Define NLP and list the common NLP tasks.
9. What is LLM? What are the stages of Building an LLM?
10. What are pretraining and finetuning? Do we need these while creating an LLM? Explain with appropriate block diagram and Explanation.
11. Prove graphically why a single-layer perceptron cannot solve the non-linearly separable XOR problem by comparing with AND & OR. How do Multi-Layer Perceptrons (MLPs) construct hyperplanes to resolve this?
12. Explain how feedforward network is trained using backpropagation algorithm.
13. Explain why Recurrent Neural Networks (RNNs) and LSTMs struggle with long text sequences (vanishing/exploding gradient and memory bottleneck problems). How do Transformers eliminate these bottlenecks?
14. What are zero short learning and few short learning and discuss it association with CNN.
15. What are zero short learning and few short learning and discuss their role in Prompt Engineering. Explain the techniques with examples
16. When do we use following explain by highlighting benefit, strength and weakness of
    a. CNN
    b. RNN
    c. LSTM
    d. Transformer

17. What is attention? Why is it important? Discuss the different type of attention mechanisms.
18. Discuss about the Architecture of Transformer. Highlight it's key components.
19. Discuss about the different variant of Transformers.
20. Define Tokenization. List and explain the types of tokenization
21. Explain Stemming and Lemmatization with examples
22. Explain how tokenization is performed using BPE.
23. What is word2vec used for? Explain one of its methods.
24. What is prompt engineering? Describe different types of prompt techniques. Define different methods to make LLM more deterministic.
25. Explain LLM Agent workflow in detail.
26. When do we use LangChain? Discuss about it's Architecture and Components
27. Describe the process of document loading and text splitting in LangChain and why it is important.
28. Explain agents and tools implementation in LangChain with a suitable example.
29. What is Retrieval Augmented Generation (RAG)? Explain different chunking strategies.
30. Explain the working of RAG with the illustration diagram.
31. What is a "hallucination" in LLM outputs? Discuss the challenges of explainability and transparency in LLM-based systems.
32. Describe multi-agent system and multi-model system.
33. What Is Transfer Learning? Why and How this works?
34. Compare and Contrast
    a. Human Brain Vs Computers
    b. AI vs Generative AI
    c. Generative Vs Discriminative Model
    d. Self-Attention Vs Multi-Head Attention
    e. BERT vs GPT
35. Write Short notes on:
    a. Chain-of-Thought Prompting
    b. LLaMA
    c. Vector Database Vs Traditional Database

36. The Office of the Prime Minister and Council of Ministers (OPMCM) in Nepal is upgrading Hello Sarkar, the national citizen grievance redressal platform, to manage thousands of multi-channel public complaints daily (e.g., driver's license delays, passport status, local infrastructure issues). The current system struggles with high response latency, manual routing bottlenecks across provincial/local authorities, and inconsistent feedback quality. To solve this, the technology team requires an API-Connected LLM Agent that can autonomously route, verify, and resolve citizen complaints by connecting to live REST APIs (e.g., Hello Sarkar tracking portal gunaso.opmcm.gov.np).
    a. What important features and design considerations must be taken into account when building an API-connected LLM agent for public governance? Design a complete architecture and illustrate the system flow in detail using a clear block diagram.
    b. b) Explain the role of components used. How would you prevent tool misfires (e.g., the LLM attempting to call a live status API for general governance policy questions)?
    c. How would you extend the agent to support multi-tool execution (e.g., chaining a grievance tracking tool with a date/calendar utility tool)? Describe how error handling (e.g., API timeouts, invalid ticket IDs) and safety alignment should be configured within the agent runtime (AgentExecutor) to ensure reliable citizen communication.